

FilterAI — Project Overview

A simple guide for the team — what we're building and how it works

1. What Is FilterAI?

FilterAI is a mobile app that helps HR teams and recruiters screen resumes automatically using AI. Instead of a person reading through 50 or 100 resumes by hand, someone uploads all the resumes plus the job description, and the app does the heavy lifting: it reads every resume, figures out which candidates actually match the job, ranks them, and even helps with next steps like sending offer letters or rejection emails.

Think of it like a smart filter — you pour in a big pile of resumes, and it hands you back the best 5-10 candidates, already scored and summarized.

2. Why This Project Is Worth Building

Screening resumes manually takes hours, and small teams often can't do it well at scale. This app solves a real, common problem — which makes it both a strong college project (it uses real, current AI techniques) and something that could genuinely be used by a real business.

3. How It Works — The Big Picture

The app has two halves that work together:

- **The mobile app (frontend)** — what the user sees and taps on their phone. Built with a tool called **Expo**, which lets us write one codebase that runs on both iPhone and Android.
- **The backend** — the "brain" running on a server, which does all the actual AI processing, talks to the database, and sends emails. Built with **Python** using a framework called **FastAPI**.

The phone app never does the heavy work itself — it just sends requests to the backend ("here are the resumes, please analyze them") and displays whatever the backend sends back.

4. Step-by-Step: What Happens When Someone Uses the App

Step 1 — Login — The user opens the app, lands on a simple welcome/landing page, and logs in (or signs up) with an email and password.

Step 2 — Upload — On the home screen, the user uploads up to 100 resume PDFs and pastes the job description they're hiring for.

Step 3 — Reading the resumes — The backend opens every PDF and pulls out the plain text from it (names, experience, skills, everything written in the resume).

Step 4 — Breaking text into pieces — That extracted text is broken into small chunks (a few sentences at a time) so it's easier for the AI to process accurately.

Step 5 — Turning text into "vectors" — Each chunk is converted into a list of numbers (called a vector) that represents its meaning. This step runs completely on our own server — it's free and doesn't use any paid AI service.

Step 6 — Storing and comparing — All these vectors are stored in a small local database. The job description is also converted into a vector, and the app mathematically compares it against every resume vector to see which resumes are the closest match. This comparison is pure math — again, completely free, no AI cost involved.

Step 7 — Shortlisting — Based on that comparison, only the top 5-10 best-matching resumes move forward. This is the key trick that makes the app efficient — we don't waste AI resources analyzing resumes that clearly aren't a good fit.

Step 8 — AI scoring — Only this shortlist is sent to an AI model (we use Groq's LLaMA model) which reads each one properly and returns a match score, a summary, a skills breakdown, and any red flags (like employment gaps).

Step 9 — Results shown to the user — The ranked candidates appear on the phone screen — the user can tap into any candidate to see full details, compare a few side-by-side, or see suggested interview questions.

Step 10 — Taking action — The user (HR/owner) decides what to do with each shortlisted candidate — send them an offer letter (AI-generated, sent by email) or reject them. Candidates who didn't make the shortlist are automatically sent a polite rejection email.

Step 11 — Reports — The user can download a single PDF report summarizing the top candidates with a comparison chart, or export the data to Excel.

5. Login & Accounts (Authentication)

Since our whole team will use this app, everyone needs their own account. When someone signs up, their password is never stored as plain readable text — it gets scrambled ("hashed") before saving, so even we can't see anyone's actual password.

When someone logs in successfully, the backend gives their phone a temporary digital "pass" (called a token). The app keeps this pass and shows it every time it asks the backend for something, proving "yes, this is a logged-in user." This is a standard, secure way apps handle logins — used by pretty much every app you already use.

6. How the App Remembers Things (State Management)

While you're using the app, it needs to "remember" a bunch of things as you move between screens — which resumes you uploaded, the job description you typed, which candidates you've shortlisted, and so on. We use a tool called **Zustand** to manage this shared memory.

Think of it as one central notebook that every screen in the app can read from and write to. So if you shortlist a candidate on one screen, that update is instantly visible on every other screen too — without needing to reload or re-fetch anything.

7. Full Tech Stack (What's Used and Why)

Part of the App	Tool Used	What It Does
Mobile App	Expo (React Native)	Builds one app that works on iPhone and Android

Remembering data on the phone	Zustand	Shared memory across all app screens
Moving between screens	React Navigation	Handles tabs, back buttons, screen transitions
Backend server	FastAPI (Python)	Handles all requests from the app, runs the AI logic
Reading resumes	PyMuPDF	Extracts text from uploaded PDF files
Breaking text into chunks	LangChain	Splits resume text into smaller, manageable pieces
Understanding meaning of text	HuggingFace Embeddings	Converts text into comparable numbers, runs locally, free
Storing & comparing text	ChromaDB	Local database that stores and matches resume data
AI scoring & writing	Groq API (LLaMA model)	Reads shortlisted resumes and generates scores, summaries, letters
Team activity history	MongoDB	Stores login accounts and a log of who analyzed what, and when
Sending emails	Gmail SMTP	Sends offer letters and rejection emails automatically
PDF reports	ReportLab	Generates the downloadable candidate summary PDF

8. The Smart Part — Why This Design Saves Cost

A lazy version of this app would send every single resume to the AI model for scoring — which is slow and adds up in cost as the number of resumes grows. Instead, our app does the first round of filtering completely on our own server, for free, using math-based comparison. Only the resumes that already look like a strong match get sent to the AI. This pattern is called **RAG** (Retrieval-Augmented Generation) — a widely used real-world technique, also used inside tools like ChatGPT, Claude, and Cursor.

This document covers the core idea and flow only. Screen-by-screen design, deployment details, and database connection setup are handled separately.